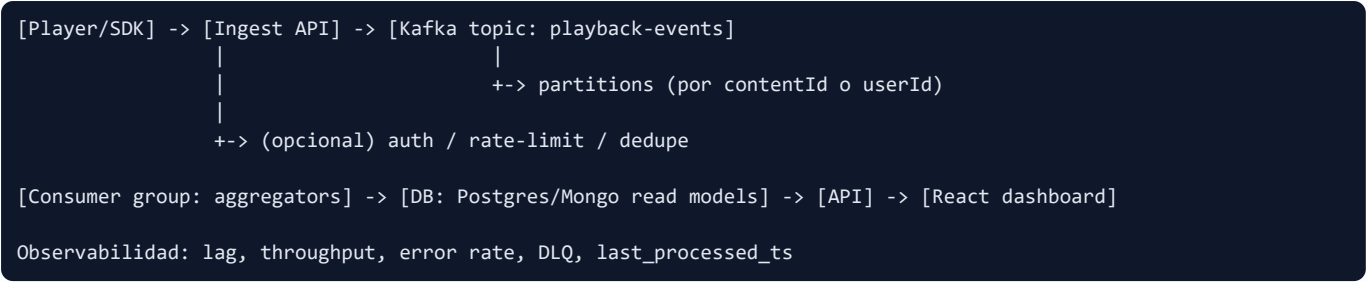


Pipeline + Kafka (Analytics) — Cheat Sheet

Qué dibujar y qué decir en 5–10 minutos. Enfoque: ingestión, colas, agregación, near-real-time y consistencia.

DIAGRAMA BASE (1 MINUTO)



Topic Partitions Consumer group At-least-once Idempotencia Lag

POR QUÉ KAFKA (VS ESCRIBIR DIRECTO A DB)

- **Desacopla** ingestión del procesamiento (picos sin tumbar DB).
- **Retries** y **reprocesamiento** (rebuild de métricas).
- **Escalado** independiente: más consumers sin tocar el SDK.
- **Orden** por key (dentro de una partition) y backpressure.

Talking point: “La cola evita que el API de ingestión sea CPU/DB-bound. La DB queda para modelos de lectura.”

QUÉ PRE-AGREGAR (READ MODELS)

Regla

- Eventos crudos (10M/día) no se consultan desde UI.
- Pre-agregar por **ventanas**: hora / día.

Ejemplo de tabla

```
-- Postgres: métricas por hora
metrics_hourly(
  hour_ts timestamptz,
  content_id text,
  country_code text,
  plays int,
  total_duration_sec bigint,
  PRIMARY KEY (hour_ts, content_id, country_code)
)
```

UI: query simple con GROUP BY sobre tabla agregada, no sobre events.

ENTREGA: NEAR-REAL-TIME (<5 MIN)

- **Lag** objetivo: p95 < 2–5 min.
- Batch corto (ej. 30–60s) o micro-batch por offsets.
- Optimizar hot paths: upserts por clave compuesta.

UI si hay atraso

- Mostrar **last_processed_ts** y badge “data delayed”.
- No “mentir”: transparencia > exactitud falsa.

SEMÁNTICAS DE ENTREGA

At-least-once (lo común)

- Puede procesar duplicados → requiere idempotencia.

Exactly-once

- Más complejo (transacciones, tooling). No siempre necesario en métricas.

Para métricas agregadas, **at-least-once + idempotencia** suele ser suficiente.

IDEMPOTENCIA (CLAVE PARA NO INFLAR PLAYS)

Opciones

- **Event ID** único (SDK lo genera) + tabla de dedupe.
- Si no hay ID: hash de (userId, contentId, timestamp, sessionId).

```
-- Dedupe simple
processed_events(
  event_id text PRIMARY KEY,
  processed_at timestampz default now()
)
```

```
-- Consumer (pseudo)
BEGIN;
INSERT INTO processed_events(event_id) VALUES ($id)
ON CONFLICT DO NOTHING;
-- si insertó => aplicar upsert a metrics_hourly
COMMIT;
```

TALKING POINTS (RESPUESTAS CORTAS)

- 1) ¿Por qué Kafka?
Desacopla ingestión del cómputo, absorbe picos, permite retries y re-procesar, y escalar consumers sin tocar el productor.
- 2) ¿Dónde agregás?
En consumers: pre-agrego por hora/día + content (+ country). La UI lee read models.
- 3) ¿Qué pasa con duplicados?
At-least-once => idempotencia: event_id + tabla processed_events (o hash).
- 4) ¿Qué mostrás si hay atraso?
last_processed_ts + indicador de delay. Transparencia al usuario.
- 5) ¿Tu experiencia real?
“No operé Kafka en prod; sí diseñé contratos API, workers/colas, idempotencia y tests de contrato. Entiendo el patrón pub/sub y lo aplico con analogías reales.”

OBSERVABILIDAD MÍNIMA

- **consumer lag** (por partition)
- throughput (events/s), p95 processing time
- error rate + retries
- DLQ (dead-letter) para poison messages
- last_processed_ts (para UI)

FALLBACK SI NO HAY KAFKA

- Webhook/API → **queue** (SQS/Bull/Redis) → worker → DB.
- Mismo diseño: dedupe, retries, backpressure, métricas.

Útil si te preguntan: “¿Cómo lo harías en MVP?”.